NumPy Laboratory Practice Questions & Solutions

Comprehensive Python Solutions for Array Manipulation, Matrix Operations, and Statistical Analysis

1. ARRAY CREATION & PROPERTIES

Q1–Q5 Code Solution:

```
import numpy as np

# 1. Create array, display shape, size, and data type
arr1 = np.array([10, 20, 30, 40, 50])
print("1. Shape:", arr1.shape, "| Size:", arr1.size, "| Dtype:", arr1.dtype)

# 2. Create 3x4 array from 1 to 12
arr2 = np.arange(1, 13).reshape(3, 4)

# 3. Zeros, ones, and identity matrix
zeros_arr = np.zeros((3, 3))
ones_arr = np.ones((3, 3))
eye_arr = np.eye(3)

# 4. Even numbers between 10 and 50
evens = np.arange(10, 51, 2)

# 5. 10 equally spaced numbers between 0 and 1
linspace_arr = np.linspace(0, 1, 10)
```

2. ARRAY ARITHMETIC

Q6–Q9 Code Solution:

```
a = np.array([10, 20, 30, 40])
b = np.array([2, 4, 5, 8])

# 6. Element-wise arithmetic
add_res = a + b
sub_res = a - b
mul_res = a * b
div_res = a / b

# 7. Square and Cube
squares = a ** 2
cubes = a ** 3

# 8. Increase elements by 15%
increased_arr = a * 1.15

# 9. Element-wise maximum and minimum
elem_max = np.maximum(a, b)
elem_min = np.minimum(a, b)
```

3. INDEXING & SLICING

Q10–Q13 Code Solution:

```
# 10. Extract elements from array 0 to 20
arr_20 = np.arange(21)
first_five = arr_20[:5]
last_five = arr_20[-5:]
idx_5_15 = arr_20[5:16]
every_second = arr_20[::2]

# 11. Replace first five elements with 100
arr_mod = arr_20.copy()
arr_mod[:5] = 100

# 12. View vs Copy demonstration
orig = np.array([10, 20, 30, 40, 50])
view_arr = orig[1:4]       # View shares memory with original
copy_arr = orig[1:4].copy() # Copy allocates separate memory
view_arr[0] = 999           # Alters orig[1] to 999; copy_arr stays unchanged

# 13. Reverse array using slicing
reversed_arr = orig[::-1]
```

4. 2-D ARRAY INDEXING & SLICING

Q14–Q16 Code Solution:

```
mat = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])

# 14. Extract row, column, element
row2 = mat[1]           # [40, 50, 60]
col3 = mat[:, 2]        # [30, 60, 90]
elem_50 = mat[1, 1]     # 50

# 15. Sub-matrix [[20,30],[50,60]]
sub_mat = mat[0:2, 1:3]

# 16. 5x5 matrix 1-25 extraction
mat_5x5 = np.arange(1, 26).reshape(5, 5)
first_row = mat_5x5[0]
last_col = mat_5x5[:, -1]
mid_elem = mat_5x5[2, 2]
top_left_3x3 = mat_5x5[:3, :3]
```

5. RESHAPING & TRANSPOSE

Q17–Q20 Code Solution:

```
# 17. Reshape 1-12 to 3x4
arr_3x4 = np.arange(1, 13).reshape(3, 4)

# 18. Reshape 1D to 2x5
arr_2x5 = np.arange(10).reshape(2, 5)

# 19. Transpose 3x4 to 4x3
arr_4x3 = arr_3x4.T

# 20. transpose() and swapaxes() on 3D array
arr_3d = np.arange(24).reshape(2, 3, 4)
transposed_3d = arr_3d.transpose()
swapped_3d = np.swapaxes(arr_3d, 0, 2)
```

6. MATRIX OPERATIONS

Q21–Q23 Code Solution:

```
m1 = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
m2 = np.array([[9, 8, 7], [6, 5, 4], [3, 2, 1]])

# 21. Element-wise addition & multiplication
m_add = m1 + m2
m_mul = m1 * m2

# 22. Matrix product using np.dot()
m_dot = np.dot(m1, m2)

# 23. Transpose dot product with original
dot_transposed = np.dot(m1.T, m1)
```

7. UNIVERSAL FUNCTIONS

Q24–Q27 Code Solution:

```
arr_ufunc = np.array([1, 4, 9, 16])

# 24. Square root
sqrt_vals = np.sqrt(arr_ufunc)

# 25. Exponential e^X
exp_vals = np.exp(arr_ufunc)

# 26. Random array calculations
rand_arr = np.random.randn(5)
rand_abs = np.abs(rand_arr)
rand_exp = np.exp(rand_arr)
rand_sqrt = np.sqrt(rand_abs) # using abs for real roots

# 27. Element-wise max of two random arrays
r1, r2 = np.random.randn(5), np.random.randn(5)
rand_max = np.maximum(r1, r2)
```

8. NP.WHERE()

Q28–Q30 Code Solution:

```
# 28. Marks below 40 replaced with 0
marks = np.array([35, 72, 48, 90, 55, 28])
marks_passed = np.where(marks < 40, 0, marks)

# 29. Income classification (>= 50,000 -> High else Low)
income = np.array([45000, 60000, 30000, 80000, 50000])
income_class = np.where(income >= 50000, 'High', 'Low')

# 30. Replace values > 30 with 100
values = np.array([10, 25, 35, 40, 5])
vals_capped = np.where(values > 30, 100, values)
```

9. STATISTICAL OPERATIONS

Q31–Q34 Code Solution:

```
stat_3x3 = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])

# 31. Total sum, mean, variance, standard deviation
total_sum = np.sum(stat_3x3)
total_mean = np.mean(stat_3x3)
total_var = np.var(stat_3x3)
total_std = np.std(stat_3x3)

# 32. Row-wise and column-wise sums
row_sums = np.sum(stat_3x3, axis=1)
col_sums = np.sum(stat_3x3, axis=0)

# 33. Student marks statistics
st_marks = np.array([65, 78, 45, 90, 88, 56, 72, 81, 39, 94])
st_mean, st_std = np.mean(st_marks), np.std(st_marks)
st_min, st_max = np.min(st_marks), np.max(st_marks)

# 34. any() and all() demonstration
bool_arr = np.array([True, False, True])
has_any = np.any(bool_arr) # True
has_all = np.all(bool_arr) # False
```

10. SORTING & UNIQUE VALUES

Q35–Q38 Code Solution:

```
# 35. Sort 10 random numbers
rand_10 = np.random.rand(10)
sorted_rand = np.sort(rand_10)

# 36 & 37. Unique elements / unique marks
repeated = np.array([10, 20, 20, 30, 40, 40, 50, 10])
unique_vals = np.unique(repeated)

# 38. Check occurrence using np.in1d()
check_items = np.array([20, 60, 30])
is_present = np.in1d(check_items, repeated)
```

11. INTEGRATED LAB QUESTIONS

Q39. Student Marks Analysis:

```
# 10 students, 3 subjects
marks_db = np.random.randint(30, 100, size=(10, 3))
totals = np.sum(marks_db, axis=1)
averages = np.mean(marks_db, axis=1)
passed_students = np.where(averages >= 50)[0]
```

Q40. Matrix Analysis:

```
mat_4x4 = np.arange(1, 17).reshape(4, 4)
mat_transposed = mat_4x4.T
r_sums = np.sum(mat_4x4, axis=1)
c_sums = np.sum(mat_4x4, axis=0)
overall_mean = np.mean(mat_4x4)
overall_max, overall_min = np.max(mat_4x4), np.min(mat_4x4)
central_2x2 = mat_4x4[1:3, 1:3]
```

Q41. Random Data Analysis:

```
obs = np.random.randn(20)
obs_mean, obs_std = np.mean(obs), np.std(obs)
obs_min, obs_max = np.min(obs), np.max(obs)
obs_sorted = np.sort(obs)
obs_clipped = np.where(obs < 0, 0, obs)
```

Q42. Matrix Operations Challenge:

```
A = np.random.randint(1, 10, (3, 3))
B = np.random.randint(1, 10, (3, 3))
elem_add = A + B
elem_mul = A * B
mat_prod = np.matmul(A, B)
A_transpose = A.T
res_max, res_min = np.max(mat_prod), np.min(mat_prod)
```

Q43. Array Manipulation Challenge:

```
arr_24 = np.arange(1, 25).reshape(4, 6)
# Extract selected rows (0,2) and cols (1,3,5)
extracted = arr_24[[0, 2]][:, [1, 3, 5]]
transposed_24 = arr_24.T
row_means = np.mean(arr_24, axis=1)
col_means = np.mean(arr_24, axis=0)
uniques = np.unique(arr_24)
```